Observer AgentInsurFlow dans Application Insights

Rattachement : E3 : C11 Monitoring.

Guide pas à pas pour ouvrir les outils d'observation d'Azure : Live Metrics, Application Map, Transaction Diagnostics, journaux KQL, tableau de bord et alertes. Il sert la recette (K9, K11) et la démo. Les procédures de diagnostic par alerte sont dans `exploitation.md` (sections 4 à 6).

1. Où sont les données

Élément	Staging	Production
Ressource Application Insights	`appi-insurflow-staging`	`appi-insurflow-production`
Espace Log Analytics (tables `AppTraces`, `AppRequests`, `AppDependencies`)	`workspace-svespucepZG6` (`LOG_ANALYTICS_WORKSPACE` du `.env`)	idem
Tableau de bord	`vigie-staging`	`vigie-production`
Groupe d'actions des alertes (e-mail `ALERTE_EMAIL`)	`ag-insurflow-staging`	`ag-insurflow-production`
Espace gestionnaire	`https://insurflow-staging.redstone-0f3be5e1.swedencentral.azurecontainerapps.io/app/`	`https://aibrokerflow.fr/app/`

Toutes ces ressources sont dans le groupe de ressources `svespuceRG`. Elles sont créées par `make vigie ENV=staging|production` (`infra/vigie.sh`). Les sept services, gateway comprise, y envoient leurs traces.

Accès : un compte Azure qui peut lire le groupe `svespuceRG`. Le rôle « Lecteur » ou « Lecteur de surveillance » suffit pour tout ce guide, et rien n'y écrit.

Ouvrir la ressource : sur `https://portal.azure.com`, taper `appi-insurflow-staging` dans la barre de recherche du haut, puis ouvrir la ressource « Application Insights ». Les outils ci-dessous sont dans son menu de gauche.

2. Produire une évaluation à observer

Les outils ne montrent quelque chose que si le système a reçu du trafic. Pour en produire :

1. Ouvrir l'espace gestionnaire de l'environnement (tableau du § 1).
2. Choisir la carte **Gestionnaire A** ou **B**, puis « Se connecter avec Microsoft ».
3. Dans **Mes dossiers**, ouvrir un dossier qui n'est pas encore décidé. La fiche du lead s'affiche en tête.
4. Cliquer sur **Lancer l'évaluation**. La frise des agents avance chaque seconde : Pièces, Analyste, Rédacteur, Actuaire.

5. Cliquer sur une étape terminée pour voir son mode, sa durée, ses tokens et son **`trace_id`**. Noter ce `trace_id` : il sert à retrouver l'évaluation au § 5.

Pour la recette K11 : **Pause** pendant l'évaluation. Elle s'arrête après l'agent en cours (`EN_PAUSE`). Ensuite, `make recette ACTION=redemarrer`, puis **Reprendre**.

Ne pas utiliser les dossiers `AIF-2026-000007` à `000009`, réservés à la CI.

3. Live Metrics : le trafic en direct

Menu de gauche, **Investiguer › Métriques temps réel** (*Live Metrics*).

- On y voit les requêtes entrantes, leur durée et les échecs, seconde par seconde, ainsi que les serveurs (répliques) connectés.
- Lancer une évaluation pendant que la page est ouverte : les requêtes de la gateway et du cluster apparaissent en quelques secondes.
- Rien n'est conservé : c'est une vue en direct seulement.

4. Application Map : qui appelle qui

Menu de gauche, **Investiguer › Carte d'application** (*Application Map*).

- Chaque bulle est un service (son `service.name`), chaque flèche un appel avec son nombre d'appels, sa durée moyenne et son taux d'échec.
- Le chemin attendu part de la gateway vers rag, underwriting-admin, le serveur MCP, le cluster d'agents et la Function de calcul. Le cluster rappelle la gateway, qui seule joint les autres services (D22, D106).
- Une flèche rouge signale des échecs : cliquer dessus, puis **Investiguer les échecs**.
- La carte se construit sur l'intervalle de temps choisi en haut (par défaut 1 h). L'élargir si l'évaluation est plus ancienne.

5. Transaction Diagnostics : une évaluation de bout en bout

Menu de gauche, **Investiguer › Recherche de transactions** (*Transaction search*).

1. Coller le `trace_id` noté au § 2 dans la zone de recherche, puis valider.
2. Ouvrir un des résultats : la vue **Détails de transaction de bout en bout** (*Transaction Diagnostics*) s'ouvre.
3. Elle montre l'arbre des appels de cette évaluation : gateway, cluster, chaque agent (span `invoke_agent`), appels au serveur MCP, à la Function de calcul, aux points de reprise (`/api/points-de-reprise/…`) et au modèle de langage. Chaque appel porte sa durée et son code de réponse.
4. Cliquer un élément pour voir ses propriétés : agent, mode, cause, réglage, tokens. Aucune donnée personnelle n'y figure, seulement des empreintes SHA-256.

Sans `trace_id`, on part du menu **Investiguer › Échecs** ou **Performances**, puis on ouvre une requête dans la liste.

6. Journaux : requêtes KQL

Deux endroits possibles, avec des noms de tables différents :

Où	Tables	Usage
Application Insights › **Surveillance** › **Journaux**	`requests`, `dependencies`, `traces`	requêtes rapides sur une ressource
Espace Log Analytics `workspace-svespucepZG6` › **Journaux**	`AppRequests`, `AppDependencies`, `AppTraces`	requêtes de `infra/vigie/kql/` (gabarits : voir `exploitation.md` § 5, « Lancer une requête d'une fiche »)

Requêtes prêtes à coller dans **Application Insights** › **Journaux** (fixer l'intervalle de temps en haut) :

```
// Une ligne par agent pour une évaluation (K9) : remplacer le trace_id
traces
| where message == "audit.agent"
| where tostring(customDimensions.trace_id) == "<trace_id>"
| project timestamp, agent = tostring(customDimensions.agent), mode =
tostring(customDimensions.mode),
          cause = tostring(customDimensions.cause), duree_ms = toint(customDimensions.duree_ms),
          tokens_entree = toint(customDimensions.tokens_entree), tokens_sortie =
toint(customDimensions.tokens_sortie)
| order by timestamp asc
```

```
// Les dernières évaluations et leur statut final
traces
| where message == "audit.evaluation"
| project timestamp, contrat = tostring(customDimensions.numero_contrat),
          statut = tostring(customDimensions.statut_final), cause =
tostring(customDimensions.cause),
          duree_ms = toint(customDimensions.duree_ms)
| order by timestamp desc
| take 20
```

```
// Points de reprise (K11, D106) : tous en 2xx, aucun 403
union requests, dependencies
| where tostring(url) has "points-de-reprise" or name has "points-de-reprise"
| summarize appels = count() by itemType, resultCode, cloud_RoleName
```

Les mêmes requêtes se lancent depuis un terminal, en lecture seule :

```
az monitor app-insights query --app appi-insurflow-staging -g svespuceRG \
  --analytics-query 'traces | where message == "audit.evaluation" | take 5'
```

7. Tableau de bord et alertes

- **Tableau de bord** : portail Azure › **Tableaux de bord** (*Dashboard*), puis `vigie-staging` ou `vigie-production`. Une tuile par question : disponibilité, évaluations par heure, p95, revue humaine par cause, coût par agent, évaluations sans décision. La lecture de chaque tuile est dans `exploitation.md` § 4.

- **Alertes** : dans la ressource Application Insights › **Surveillance** › **Alertes**, on voit les règles AL-1 à AL-10 et celles qui sont déclenchées. Chaque alerte envoie un e-mail par le groupe d'actions `ag-insurflow-<env>`. La fiche de chaque alerte est dans `exploitation.md` § 6.

8. Si rien n'apparaît

- **Délai d'ingestion** : compter 2 à 5 minutes entre l'évaluation et son apparition dans les journaux et la recherche de transactions. Live Metrics, lui, est immédiat.
- **Intervalle de temps** : vérifier en haut de page qu'il couvre l'heure de l'évaluation.
- **Bonne ressource** : vérifier que staging et production ne sont pas confondus.
- **Quels services émettent ?** Cette requête liste les services présents sur 7 jours. Le cluster s'appelle `agentic-cluster` (`OTEL_SERVICE_NAME` de sa Container App) :

  ```
  union requests, traces, dependencies, performanceCounters
  | where timestamp > ago(7d)
  | summarize n = count(), dernier = max(timestamp) by cloud_RoleName, itemType
  ```

  Sans évaluation, le cluster n'émet que ses compteurs de performance (`performanceCounters`) : il n'a ni requête, ni `audit.*`, ni span d'agent. Constat du 30/10 (#176) : les 2 évaluations réelles de staging ont produit les spans du cluster (`POST /evaluations/{numero_contrat}`, `invoke_agent`), mais aucune ligne `audit.*` : le journal `insurflow.audit` restait au niveau WARNING, car le gestionnaire posé par Azure Monitor empêchait de régler INFO. Après le correctif, une évaluation de contrôle (§ 2) doit faire apparaître `audit.agent` et `audit.evaluation` : sinon, rouvrir #176.
- **Service absent de la carte** : il n'a pas émis de trace dans l'intervalle, ou sa variable `APPLICATIONINSIGHTS_CONNECTION_STRING` manque. Voir `exploitation.md` § 5.

9. Pourquoi cet outil, et son accessibilité

Choix. Application Insights et Log Analytics, restitués par un tableau de bord du portail Azure.

Critère	Application Insights + portail Azure	Grafana managé
Déjà dans le projet	Oui : les services y exportent déjà leurs traces, dans la même région et le même abonnement	Non : un service de plus à provisionner, à sécuriser et à payer
Authentification	Entra ID, comme le reste du projet	À raccorder à Entra ID
Temps réel	Live Metrics, immédiat ; journaux en 2 à 5 minutes	Dépend de la même source de données
Alertes par e-mail	Oui, avec les règles AL-1 à AL-10	Oui, à reconfigurer
Contrainte du projet : un seul développeur	Rien de plus à exploiter	Une brique de plus à maintenir

Limite assumée. Le portail est un outil d'exploitation, réservé aux personnes ayant accès à Azure : ni les gestionnaires ni les souscripteurs n'en ont besoin.

Accessibilité dans le choix de l'outil. Microsoft publie des rapports de conformité d'accessibilité (WCAG, EN 301 549) pour ses services. Le niveau précis du portail n'a pas été vérifié ici : le rapport du portail Azure reste à consulter et à joindre. Un graphique peut rester difficile à lire au lecteur d'écran. Le projet ne dépend donc pas des seuls graphiques :

Besoin	Moyen sans graphique
Lire une valeur de tuile	La requête KQL correspondante (`infra/vigie/kql/`) rend un **tableau**, parcourable au clavier et exportable en CSV
Être prévenu	L'alerte arrive par **e-mail**, un texte lisible dans le client de messagerie de chacun
Réagir à une alerte	Les fiches de `exploitation.md` § 6 sont du texte structuré, sans information portée par la seule couleur